October 20, 2020

# PostgreSQL | pg_rewind changes in PostgreSQL 13

There have been a lot of articles about new features in the recent PostgreSQL 13 release, but one set of improvements which seems to have escaped much attention is in `pg_rewind`, which has gained some additional useful functionality deserving of closer examination.

## New option "-R /--write-recovery-conf"

As the name implies, this option causes `pg_rewind` to write recovery (replication) configuration (and additionally the "`standby.signal`" file) after rewinding a server, in exactly the same way that pg_basebackup does. This is useful for example when restoring a former primary to become a standby. Note that it can only be used in conjunction with the option `--source-server`, and (unless the `--dry-run` option is supplied) that the configuration will be written regardless of whether a rewind is needed.

As replication configuration via the `recovery.conf` file was removed in PostgreSQL 12, the generated replication configuration is actually appended to `postgresql.auto.conf`, which is the one configuration file which is guaranteed to be in a known location (the data directory) and

which will be read last when PostgreSQL starts up, ensuring any prior replication configuration is overridden by whatever `pg_rewind` has written.

Currently the only replication configuration item actually written is `primary_conninfo` - be aware that this is generated from a blend of the connection information passed in `--source-server` and the default parameters generated by libpq, which may be different to the `primary_conninfo` string you would normally generate and will look something like this:

```
primary_conninfo = 'user=postgres passfile=''/var/lib/pgsql/.pgpass'' channel_binding=prefer host=node1 port=5432
sslmode=prefer sslcompression=0 ssl_min_protocol_version=TLSv1.2 gssencmode=disable krbsrvname=postgres target_session_attrs=any'
```

Note that currently replication slot configuration (`primary_slot_name`) cannot be generated by `pg_rewind`.

Also be aware that if attempting to rewind a standby which was not shut down properly, the `standby.signal` file must be removed otherwise `pg_rewind` will attempt to start the server in single user mode to run recovery, which cannot be done with replication configuration in place. See also the section "Automatic crash recovery" below.

This option was added in commit 927474ce

# New option "-c/--restore-target-wal"

`pg_rewind` requires that the server to be rewound (the "target server") contains WAL files reaching back to the point of divergence, which can result in situations like this:

```
$ pg_rewind -D /var/lib/pgsql/data --source-server='host=node1 dbname=postgres user=postgres port=5432'
pg_rewind: servers diverged at WAL location 0/30005C8 on timeline 1
(... snip log output from postgres starting up in single user mode ...)
pg_rewind: error: could not open file "/var/lib/pgsql/data/pg_wal/000000010000000000000002": No such file or directory
pg_rewind: fatal: could not find previous WAL record at 0/2000100
```

Previously, it would have been necessary to manually retrieve any missing files, however from PostgreSQL 13 `pg_rewind` provides the `-c`/`--restore-target-wal` option, which uses the local server's `restore_command` to fetch required files.

Assuming `restore_command` is correctly defined and the required WAL files are available in the archive, with `c`/`--restore-target-wal` the preceding scenario should now succeed:

```
$ pg_rewind -D /var/lib/pgsql/data --source-server='host=node1 dbname=postgres user=postgres port=5432' --restore-target-wal
pg_rewind: servers diverged at WAL location 0/30005C8 on timeline 1
pg_rewind: rewinding from last common checkpoint at 0/2000060 on timeline 1
pg_rewind: Done!
```

Note that there is no mention of whether any missing files were retrieved with via the `restore_command`; this is only shown in the (extremely verbose) `--debug` output mode. Any errors will however be reported, e.g.:

```
$ pg_rewind -D /var/lib/pgsql/data --source-server='host=node1 dbname=postgres user=postgres port=5432' --restore-target-wal
pg_rewind: servers diverged at WAL location 0/30005C8 on timeline 1
cp: cannot stat "/var/lib/wal-archive/000000010000000000000002": No such file or directory
pg_rewind: error: could not restore file "000000010000000000000002" from archive
pg_rewind: fatal: could not find previous WAL record at 0/2000100
```

(Disclaimer: the use of `cp` in `restore_command` is for demonstration purposes only and is not recommended for real-world scenarios.)

This option was added in commit a7e8ece4.

# Automatic crash recovery

`pg_rewind` can only operate on a cleanly shut down PostgreSQL instance, otherwise it won't be able to correctly determine which (if any) changes need to be applied. in PostgreSQL 12 and earlier, this meant starting the server (preferably in single user mode) so it can perform crash recovery, before stopping it again and executing `pg_rewind`:

```
$ pg_rewind -D /var/lib/pgsql/data --source-server='host=node1 dbname=postgres user=postgres port=5432'
pg_rewind: fatal: target server must be shut down cleanly
```

With PostgreSQL 13, when `pg_rewind` detects the server was not shut down cleanly, it will automatically start the server (in single user mode) to ensure crash recovery is performed. This simplifies the process of recovering a crashed server.

```
$ pg_rewind -D /var/lib/pgsql/data --source-server='host=node1 dbname=postgres user=postgres port=5432'
pg_rewind: executing "/usr/bin/pgsql/postgres" for target server to complete crash recovery
(... snip log output from postgres starting up in single user mode ...)
pg_rewind: servers diverged at WAL location 0/30005D0 on timeline 1
pg_rewind: rewinding from last common checkpoint at 0/2000060 on timeline 1
pg_rewind: Done!
```

Be aware however that unless one of the options `--no-ensure-shutdown` or `--dry-run` is provided, `pg_rewind` will perform automatic crash recovery before performing any other checks.

If `--dry-run` is provided, `pg_rewind` will indicate it is going to perform crash recovery, but not actually do so, and will exit with:

```
$ pg_rewind -D /var/lib/pgsql/data --source-server='host=node1 dbname=postgres user=postgres port=5432' --dry-run
pg_rewind: executing "/usr/bin/pgsql/postgres" for target server to complete crash recovery
pg_rewind: fatal: target server must be shut down cleanly
```

Also, as PostgreSQL's single user mode cannot be used when the server is in recovery, the presence of a `standby.signal` file (in the case of a former standby) will cause the operation to fail and `pg_rewind` to abort:

```
$ pg_rewind -D /var/lib/pgsql/data --source-server='host=node1 dbname=postgres user=postgres port=5432'
pg_rewind: executing "/usr/bin/pgsql/postgres" for target server to complete crash recovery
....
[2020-10-20 23:47:21 JST]    FATAL:  0A000: standby mode is not supported by single-user servers
....
pg_rewind: error: postgres single-user mode in target cluster failed
pg_rewind: fatal: Command was: "/usr/bin/pgsql/postgres" --single -F -D "/var/lib/pgsql/data" template1 < "/dev/null"
```

In this case `standby.signal` would need to be manually removed before, and replaced after the rewind operation.

This functionality was added in commit 5adafaf1.
Posted at 10:31 AM

---